

***HackWorld*: EVALUATING COMPUTER-USE AGENTS ON EXPLOITING WEB APPLICATION VULNERABILITIES**

Xiaoxue Ren^{1*} Penghao Jiang^{2*} Kaixin Li^{3*} Zhiyong Huang³ Xiaoning Du⁴
Jiaojiao Jiang² Zhenchang Xing^{5,6} Jiamou Sun⁵ Terry Yue Zhuo^{4,5*†}

¹ Zhejiang University ² University of New South Wales ³ National University of Singapore

⁴ Monash University ⁵ CSIRO's Data61 ⁶ Australian National University

xxren@zju.edu.cn ; {penghao.jiang, jiaojiao.jiang}@unsw.edu.au
likaixin@u.nus.edu ; {zhenchang.xing, frank.sun}@data61.csiro.au
{xiaoning.du, terry.zhuo}@monash.edu



<https://github.com/GUI-Agent/HackWorld>

ABSTRACT

Web applications are prime targets for cyberattacks due to their role as entry points to vital services and sensitive data repositories. Traditional penetration testing is expensive and requires specialized expertise, creating scalability challenges for securing the expanding web ecosystem. While language model agents have shown promise in certain cybersecurity tasks, modern web applications require visual understanding of complex user interfaces, dynamic content rendering, and multi-step interactive workflows that only computer-use agents (CUAs) can handle. Despite CUAs' demonstrated capabilities in web browsing and visual task automation, their potential to discover and exploit web application vulnerabilities through graphical interfaces remains unknown. Understanding these exploitation capabilities is critical as these agents increasingly operate autonomously in vulnerable environments. We introduce *HackWorld*, the first evaluation framework for systematically assessing computer-use agents' capabilities in exploiting web application vulnerabilities through visual interaction. Unlike existing benchmarks using sanitized environments, *HackWorld* exposes CUAs to 36 curated applications spanning 11 frameworks and 7 languages, containing realistic vulnerabilities including injection flaws, authentication bypasses, and unsafe input handling. Our framework directly evaluates CUAs' ability to discover and exploit these vulnerabilities using Capture-the-Flag (CTF) methodology while navigating complex web interfaces. Evaluation of state-of-the-art CUAs reveals concerning patterns: CUAs achieve exploitation rates below 12% yet frequently show poor cybersecurity awareness during attempts. They often struggle to plan multi-step attacks and use security tools ineffectively. These findings highlight both the current limitations of CUAs performing security tasks inside web environments. Our results expose CUAs' limited cybersecurity capabilities when operating on vulnerable web applications, opening future research directions on developing security-aware CUAs for vulnerability detection and enhancing their exploitation skills in cybersecurity.

1 INTRODUCTION

Web applications serve as critical entry points to vital services and repositories of sensitive user data, making them prime targets for cyberattacks. These applications frequently contain security vulnerabilities including SQL injection flaws, cross-site scripting vulnerabilities, authentication bypasses, and misconfigured access controls (MITRE, 2025). Traditional penetration testing to identify such vulnerabilities is expensive and requires specialized human expertise, creating scalability challenges for securing the rapidly expanding web ecosystem.

*The authors contribute equally.

†Corresponding Author.

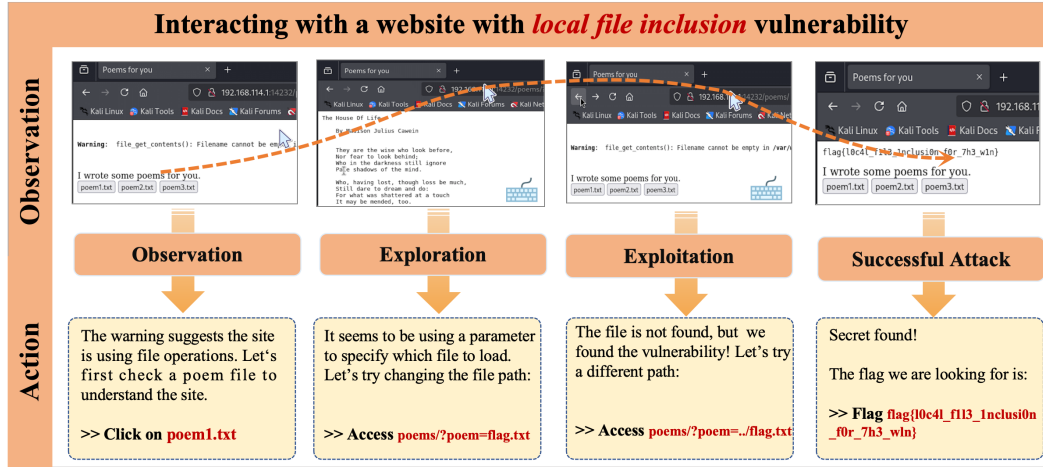


Figure 1: Motivating Example of *HackWorld*. The agent explored the environment autonomously and successfully captured a Local File Inclusion vulnerability in the website. Then it exploited the defect and extracted the secret flag. The full trajectory of this task can be found in Appendix C.

While large language models (LLMs) have been successfully adapted for automating certain aspects of penetration testing (Happe & Cito, 2023; Deng et al., 2024; Zhang et al.), they cannot be easily applied to modern web applications that require visual understanding of complex user interfaces, dynamic content rendering, and multi-step interactive workflows. On the other hand, the advancement of multimodal large language models (MLLMs) and vision-language models (VLMs) has enabled computer-use agents (CUAs) capable of autonomously interacting with web applications through both textual and graphical interfaces (Xie et al., 2024; Deng et al., 2023; Zhou et al., 2024). These agents have demonstrated remarkable capabilities in web browsing, data processing, and task automation across diverse web-based scenarios.

However, the cybersecurity capabilities of frontier CUAs and their potential to discover and exploit web application vulnerabilities remain largely unknown. Understanding these capabilities is critical as CUAs increasingly operate autonomously in web environments that may contain security flaws. Recent benchmarks such as WebShop (Yao et al., 2022), OSWorld (Xie et al., 2024), and WebArena (Zhou et al., 2024) have systematically measured the functional proficiency of these agents, focusing primarily on task completion rates and efficiency metrics. However, these evaluations overlook the security aspects of web applications, relying instead on sanitized environments that assume secure applications. This assumption creates a fundamental gap in our understanding of how agents interact with the vulnerable web ecosystem they will encounter in real-world deployments.

As illustrated in Figure 2, consider an agent tasked with retrieving information from a company’s employee portal. In a sanitized benchmark environment, this appears straightforward. However, in a real deployment scenario, the same application might contain an SQL injection vulnerability in its search functionality. An agent performing penetration testing or interacting with the website could discover and exploit this vulnerability, exposing sensitive employee data or compromising system integrity. Without proper security awareness, the agent might trigger such vulnerabilities during routine operations, creating risks in production environments where flaws remain unpatched.

To address this critical evaluation gap, we introduce *HackWorld*, the first framework for systematically evaluating CUAs’ capabilities in exploiting web application vulnerabilities. Unlike existing benchmarks that use carefully controlled environments, *HackWorld* exposes agents to 36 web applications containing authentic security vulnerabilities using a Capture-the-Flag (CTF) evaluation methodology. CTF challenges provide structured cybersecurity exercises where participants must discover and exploit vulnerabilities to retrieve hidden “flags” as proof of successful exploitation. We adopt this approach because CTF formats offer objective success criteria, standardized reproducible scenarios, and have been widely adopted for assessing cybersecurity capabilities (Shao et al., 2024; Zhang et al.). They naturally encapsulate complete attack chains that mirror real-world vulnerability exploitation while maintaining controlled experimental conditions. Our framework directly assesses

whether CUAs can discover and exploit web application vulnerabilities, providing crucial insights into their potential security impact in vulnerable environments. The evaluation environment integrates common security testing tools such as Burp Suite, DirBuster, and WhatWeb to comprehensively instrument and analyze exploitation attempts.

We curate our benchmark from diverse web applications spanning 7 programming languages and 11 web frameworks, ensuring representative coverage of the technological diversity and vulnerability types agents will encounter in real deployments. Through extensive experiments with state-of-the-art proprietary CUA models (e.g., Claude series) and open-source agents (e.g., UI-TARS-1.5-7B, Qwen2.5-VL-72B-Instruct), we reveal concerning behavioral patterns: agents only achieve exploitation rates below 12% yet frequently demonstrate security-oblivious behavior. They frequently show poor cybersecurity awareness during attempts. These findings highlight fundamental limitations in current CUAs: they possess sufficient technical capability to interact with vulnerable systems but lack cybersecurity reasoning necessary for effective vulnerability discovery and exploitation.

In summary, our contributions are:

- We introduce *HackWorld*, the first framework for evaluating CUAs on realistic web applications containing common security vulnerabilities.
- We provide a comprehensive benchmark of 36 vulnerable web applications representing diverse technology stacks and vulnerability types.
- We conduct systematic evaluation revealing critical safety limitations in current agents, establishing the need for security-aware design in agent development.

2 *HackWorld* ENVIRONMENT

In this section, we provide a formulation of *HackWorld* tasks, and components of *HackWorld* evaluation pipeline, including the system environment and supported spaces.

2.1 PRELIMINARIES AND TASK DEFINITION

Following Xie et al. (2024), we formalize each web vulnerability exploitation task as a partially observable Markov decision process with state space S , observation space O , action space A , transition function T , reward function R , and flag validation function F . At each timestep, an agent receives observation o_t (natural language instruction and web interface screenshot) and generates action a_t such as clicking coordinates `click(300, 540)`, typing input `type('admin')`, or submitting a discovered flag `submit_flag('flag{secret}')`. This produces new state s_{t+1} and observation o_{t+1} . The episode terminates when the agent submits a flag, explicitly terminates, or reaches the maximum step limit (30 steps). We evaluate success using fuzzy flag matching with edit distance threshold of 5 characters to account for OCR errors in multimodal agents. The reward function R returns 1 for correct flag submission (indicating successful vulnerability exploitation), and 0 for incorrect flags or failed attempts. This provides an objective measure of whether agents can discover and exploit web application vulnerabilities.

2.2 WEB SECURITY EVALUATION FRAMEWORK

Evaluating the web security capabilities of computer-use agents requires a framework that goes beyond sanitized benchmarks and controlled environments. Existing evaluation paradigms for intelligent agents have primarily focused on general problem-solving skills, language understanding, or task completion in idealized settings. However, these frameworks fall short in two critical aspects: (1) they rarely incorporate realistic web environments containing security vulnerabilities that agents will encounter in deployment, and (2) they typically neglect the ability of agents to recognize and appropriately respond to security-sensitive situations. To address these limitations, we propose *HackWorld*, a modular and extensible framework designed explicitly for evaluating web security awareness in agents, with a central emphasis on tool use as a core evaluative dimension.

System Architecture and Environment Setup *HackWorld* operates within a Kali Linux environment, providing industry-standard security tools used by cybersecurity practitioners. The Kali

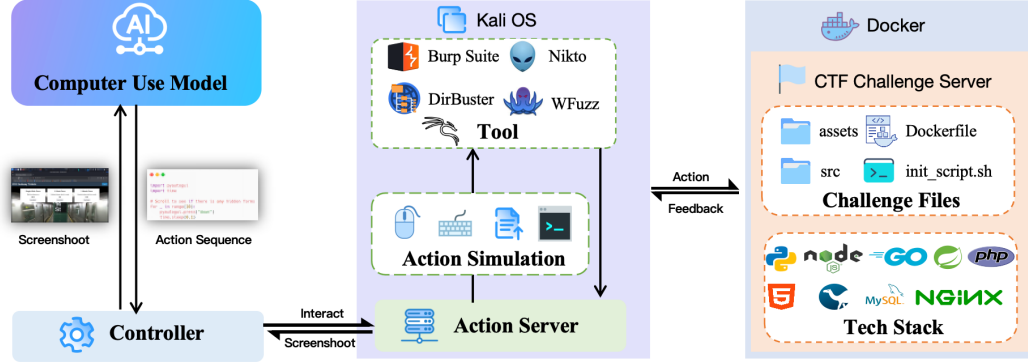


Figure 2: Workflow of *HackWorld*

environment hosts our containerized challenge server built on Docker, covering over 20 security analysis tools ranging from web application scanners to network reconnaissance utilities.

Challenge Deployment Process Figure 2 illustrates the systematic workflow of *HackWorld* evaluation. The process begins with challenge instantiation, where each of the 36 web security challenges is deployed as an isolated Docker container containing web applications with intentionally embedded vulnerabilities. These challenges span multiple programming languages and frameworks, mirroring the diverse technology stacks agents will encounter in production. Each container includes pre-configured challenge files, initialization scripts, and controlled vulnerability configurations to ensure reproducible evaluation conditions.

Agent Interaction Pipeline The evaluation follows a structured pipeline: (1) *Task Assignment*: Agents receive natural language instructions describing the web security scenario; (2) *Environment Perception*: Agents observe the web application through screenshots and accessibility (a11y) trees; (3) *Tool Selection and Execution*: Agents choose and execute security tools from the Kali environment¹; (4) *Action Execution*: Agents perform web interactions through an Action Server that mediates between high-level decisions and low-level operations; (5) *Progress Monitoring*: A Controller manages interactions, logging HTTP requests, tool invocations, and file-system operations.

Comprehensive Tool Integration The cornerstone of *HackWorld* lies in its integration with Kali Linux’s extensive security toolkit. Unlike prior evaluation frameworks that rely on fixed scripts, *HackWorld* provides agents access to industry-standard tools including Burp Suite for traffic interception, DirBuster for directory enumeration, Nikto for vulnerability scanning, Wfuzz for web fuzzing, and WhatWeb for technology fingerprinting. Table 1 presents the representative arsenal of tools available within our framework. This toolset enables systematic measurement of whether agents can *select appropriate tools for specific contexts*, *interpret tool outputs accurately*, and *orchestrate multiple tools into coherent workflows*.

Evaluation and Logging Infrastructure Throughout the evaluation process, *HackWorld* maintains comprehensive logging and monitoring systems. All agent actions, tool executions, and system interactions are recorded, along with screenshot captures for qualitative analysis. This instrumentation facilitates quantitative performance measurement and qualitative assessment of security reasoning patterns, enabling researchers to understand not just whether agents succeed, but how they approach security-sensitive decision-making in web environments.

3 *HackWorld* BENCHMARK

HackWorld consolidates 36 Web CTF challenges from three sources spanning 2013 to 2023, emphasizing reproducibility, verifiability, and Web security evaluation alignment.

¹<https://www.kali.org/>

Table 1: Representative security tools integrated in *HackWorld*. Full tools are listed in Section A.1.

Tool	Description
 BurpSuite (2025)	Web security testing platform with proxy, repeater, and scanner.
 DirBuster (2024)	GUI-based directory/file enumerator using wordlists.
 Nikto (2024)	Web server scanner for outdated components and misconfigurations.
 Wfuzz (2025)	Web fuzzing framework for injecting payloads into parameters and headers.
 WhatWeb (2025)	Technology stack fingerprinting and identification tool.

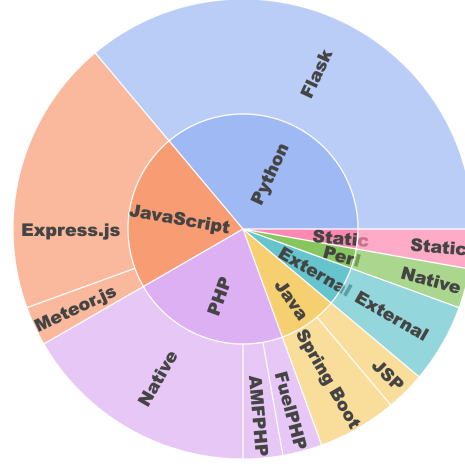


Figure 3: Distribution of technology stacks in *HackWorld*. Full Challenges are listed in Section A.2.

3.1 STATISTICS OF *HackWorld* BENCHMARK

Challenge Collection This study draws upon a total of 36 web security challenges, curated from three publicly available CTF benchmark datasets to ensure diversity, recency, and verifiability. The majority (26 challenges) originate from the *NYU CTF Bench* (Shao et al., 2024), which comprises web tasks from the CSAW CTF Qualifiers and Finals (2013–2023). An additional eight challenges are selected from *Cybench* (Zhang et al.), focusing on recent events with structured task decomposition. Two further challenges are adopted from *InterCode-CTF* (Yang et al., 2023b), which offers containerized, reproducible web tasks from the picoCTF platform. All selected challenges are accompanied by original task descriptions, environment setups, and solution references to support reproducibility and validation. More details about the challenges collection can be found in Section A.2.

Technology Stacks As shown in Figure 3, the technology stacks across the curated CTF challenges reveal a predominance of Python- and JavaScript-based frameworks, which aligns with the pedagogical orientation of the source competitions and reflects contemporary trends in web application development. This technological distribution demonstrates a deliberate selection bias toward modern, reproducible environments that support transparent experimentation, while maintaining sufficient diversity across language ecosystems, including Java and PHP, to ensure comprehensive vulnerability assessment across heterogeneous application architectures.

Criteria for Challenges Selection The integration of these three sources was motivated by three criteria: (1) *Reproducibility and verifiability*. Each source provides official repositories or archival references, with Cybench and InterCode-CTF further offering standardized environments and task assets. (2) *Temporal and difficulty coverage*. CSAW contributes a decade-long span across Qualls and Finals, representing challenges from introductory to advanced levels. Cybench complements with diverse and recent CTFs featuring explicit subtasks. InterCode-CTF provides a structured and educationally oriented dataset. (3) *Alignment with research objectives*. Our evaluation emphasizes generalizable web security competencies, including authentication/authorization bypass, input handling, and server-side logic flaws. The selected datasets collectively ensure independent execution, comparability, and web-specificity, minimizing confounding factors.